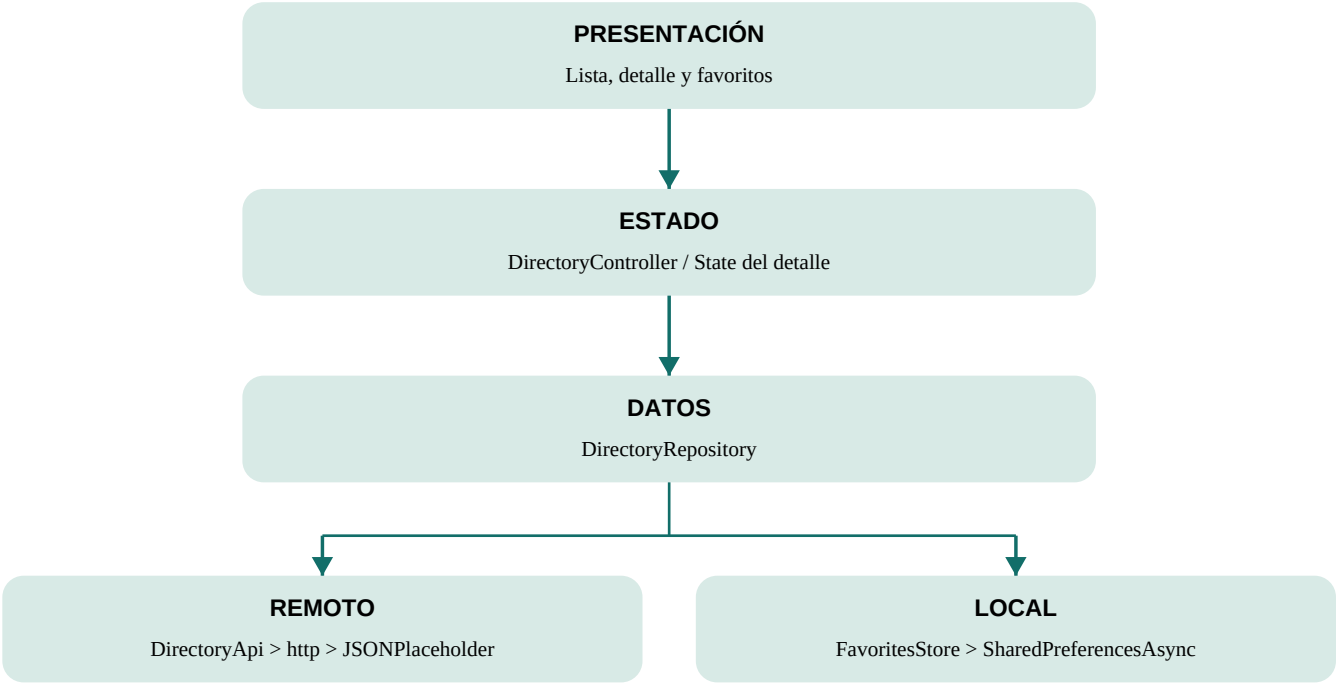
Tarea 5

Directorio de usuarios con JSONPlaceholder
Flutter / Dart • Android • Persistencia local

Objetivo y resultado

Se desarrolla una app que consulta usuarios de JSONPlaceholder, muestra su nombre, correo y empresa, abre sus datos y publicaciones y permite guardar favoritos en el dispositivo. La interfaz distingue carga, éxito, error, ausencia de red y listas vacías. El código fuente se entrega completo junto con instrucciones de ejecución.

Diagrama de arquitectura



Se utiliza una **arquitectura por capas**. La presentación dibuja las pantallas; el controlador gestiona usuarios, favoritos y conectividad; los servicios de datos realizan HTTP y almacenamiento. El detalle mantiene el estado de sus publicaciones en su State y consulta el servicio remoto del repositorio. Por eso no se describe como MVVM estricto.

DirectoryRepository agrupa DirectoryApi y FavoritesStore. Las pantallas no construyen URLs ni serializan los favoritos. Los modelos User y Post representan los datos consumidos. ListenableBuilder redibuja la lista y el icono de favorito cuando cambia el controlador.

Repositorio de la entrega

https://github.com/LAL0098/Moviles_Utesa

El repositorio contiene el proyecto en Tarea5/, los PDF en informes/ y las capturas reales en capturas/. Incluye instrucciones de ejecución, pruebas y el registro de validación.

Ciclo de vida y comportamiento

Carga de datos, persistencia y manejo de errores

Método / evento	Relación con la carga de datos
initState de la lista	Crea el controlador, lee favoritos, consulta conectividad, se suscribe a sus cambios y carga GET /users una vez por instancia.
build	Dibuja el estado actual mediante ListenableBuilder. No hace peticiones, evitando descargas repetidas en cada reconstrucción.
initState del detalle	Solicita GET /posts?userId={id} del usuario seleccionado.
mounted	Antes de setState tras una operación asíncrona del detalle, comprueba que la pantalla siga montada.
dispose	La lista libera el controlador, cancela la suscripción y cierra el cliente HTTP. El controlador evita notificar después de eliminarse.
Reconexión / reintento	Al pasar de ausencia de red a una red disponible se actualiza la lista. Actualizar y Reintentar permiten repetir solicitudes fallidas.

Al navegar al detalle se conserva la pantalla de lista. Si el sistema termina el proceso, los usuarios y publicaciones en memoria se pierden; al reiniciar se recuperan los favoritos del almacenamiento local y se vuelve a solicitar la lista. No se promete conservar el estado efímero a través de la muerte del proceso.

Datos sin conexión

FavoritesStore conserva el JSON completo de cada usuario favorito bajo la clave favorites_v1. Se pueden abrir nombre, correo, empresa, teléfono, web y dirección sin consultar la API. No basta guardar solamente los identificadores: sin red no sería posible recuperar sus datos.

Las publicaciones requieren Internet y no se guardan en esta implementación. Cuando no hay conexión, el detalle informa esta limitación y ofrece Reintentar. Los datos públicos de 10 usuarios constituyen un conjunto pequeño y no crítico; para una base mayor se usaría almacenamiento estructurado.

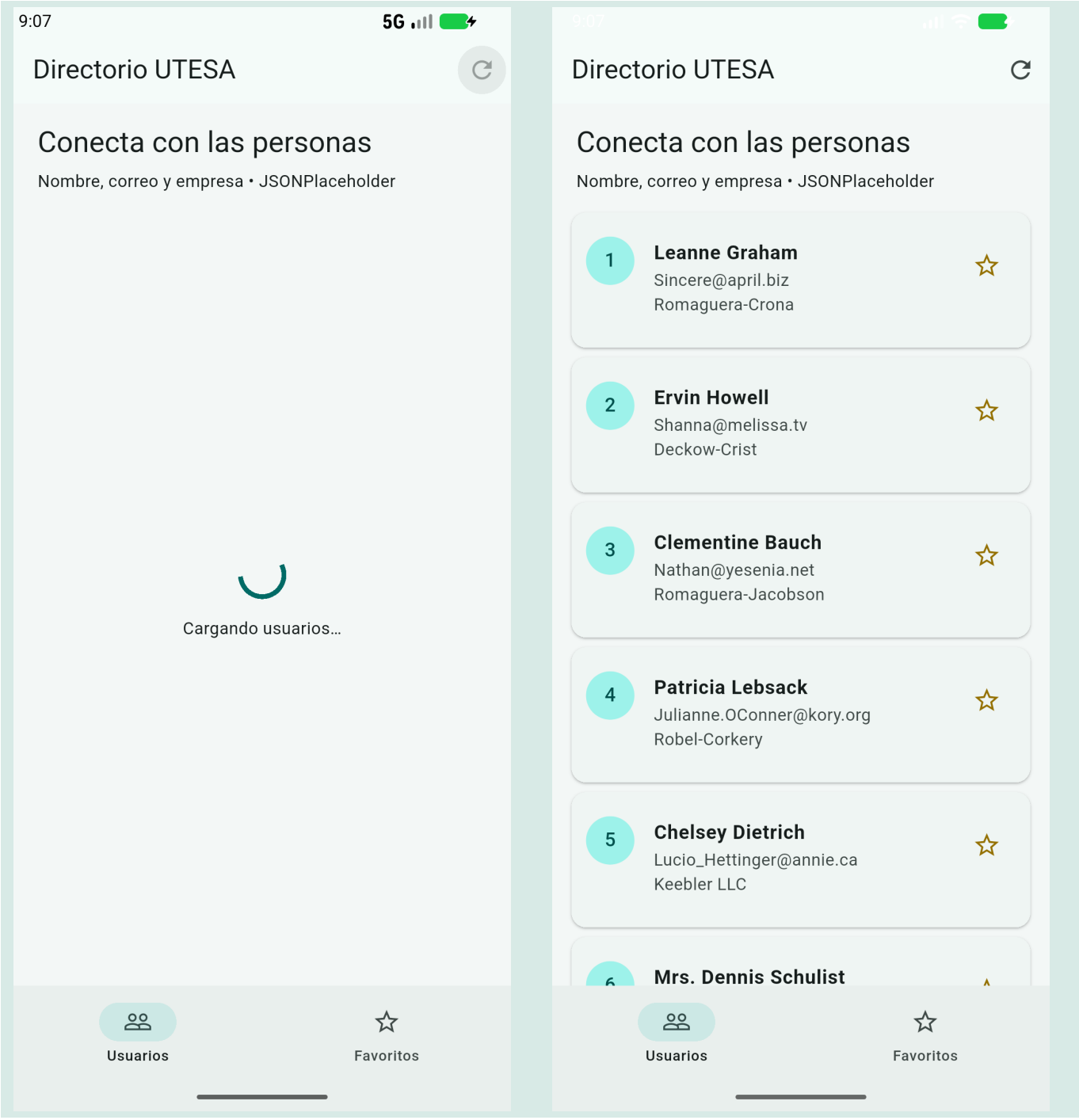
Errores y permisos

Cada solicitud tiene un límite de 15 segundos. Se comprueba el código 200 antes de deserializar. Los fallos HTTP, de conexión o de formato se convierten en mensajes visibles. Un fallo al escribir favoritos conserva el estado anterior y muestra un aviso. La detección de Wi-Fi no garantiza Internet: también se manejan los errores de la solicitud real.

```
<!-- android/app/src/main/AndroidManifest.xml -->
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

Estados de la interfaz

Capturas reales del emulador Android

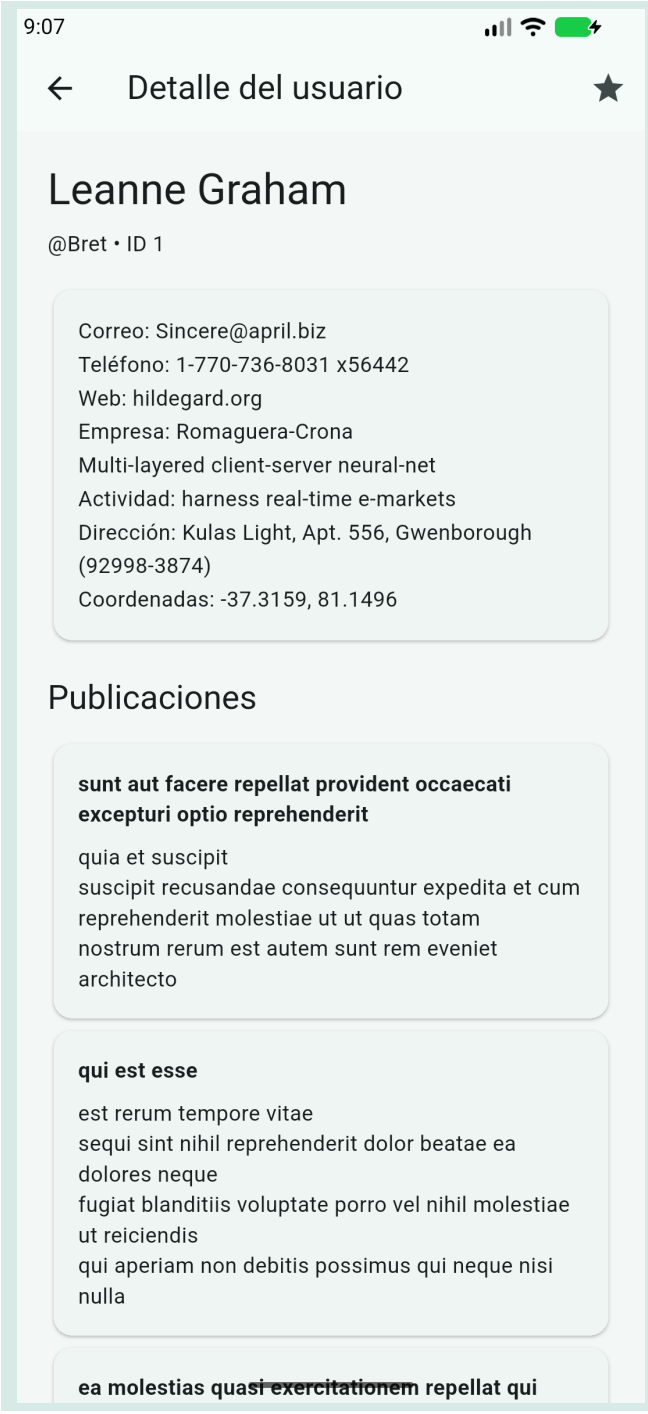


Carga: indicador durante GET /users.

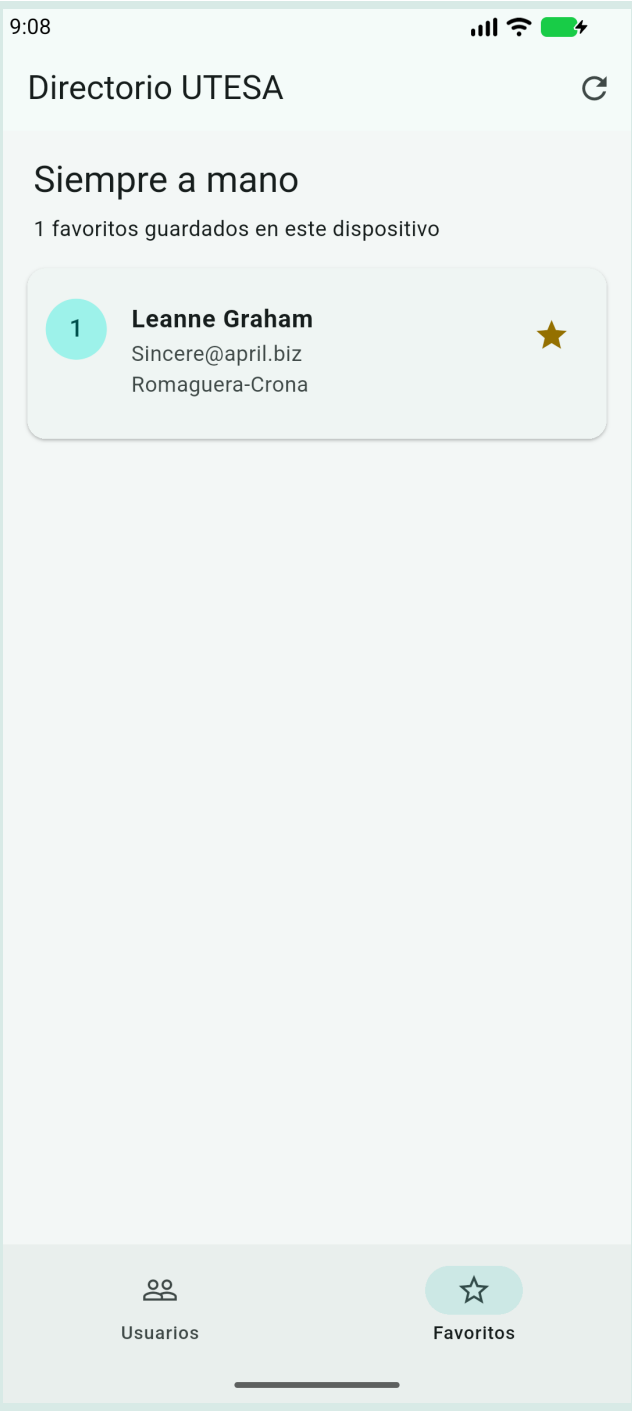
Éxito: nombre, correo y empresa.

Detalle y favoritos

Navegación y datos persistentes



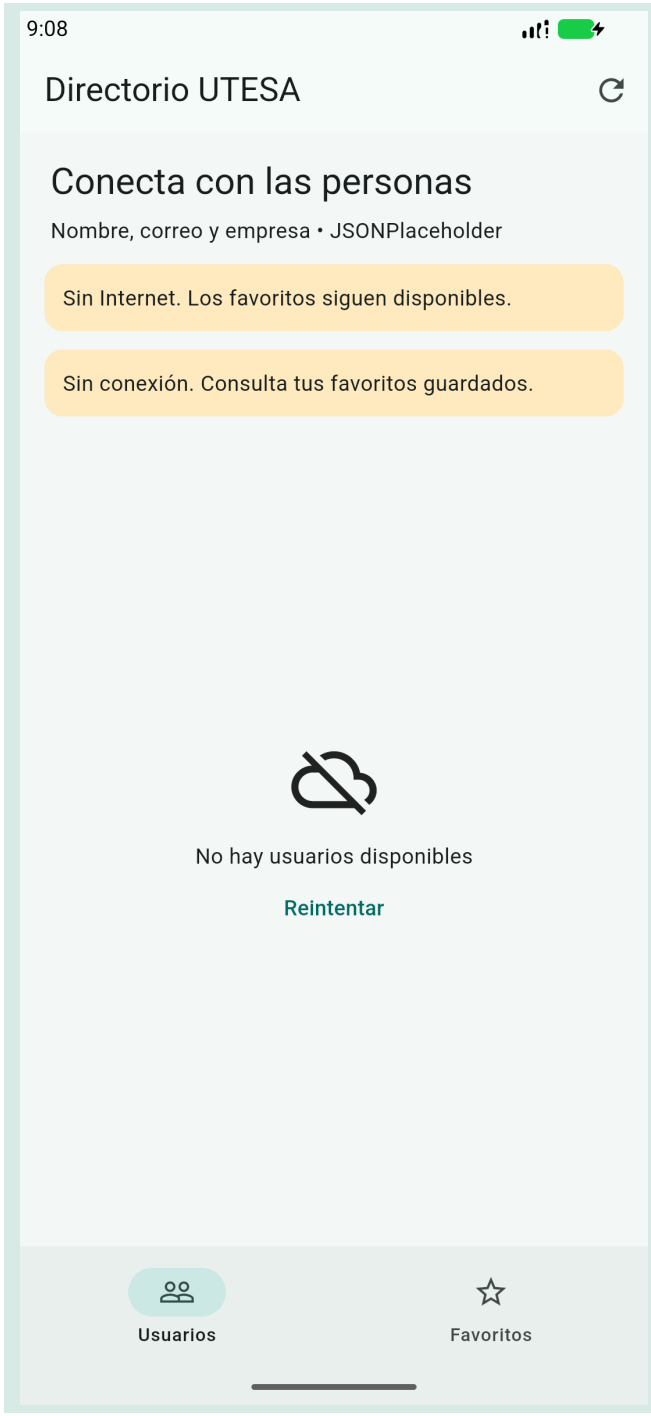
Detalle: datos y publicaciones del usuario.



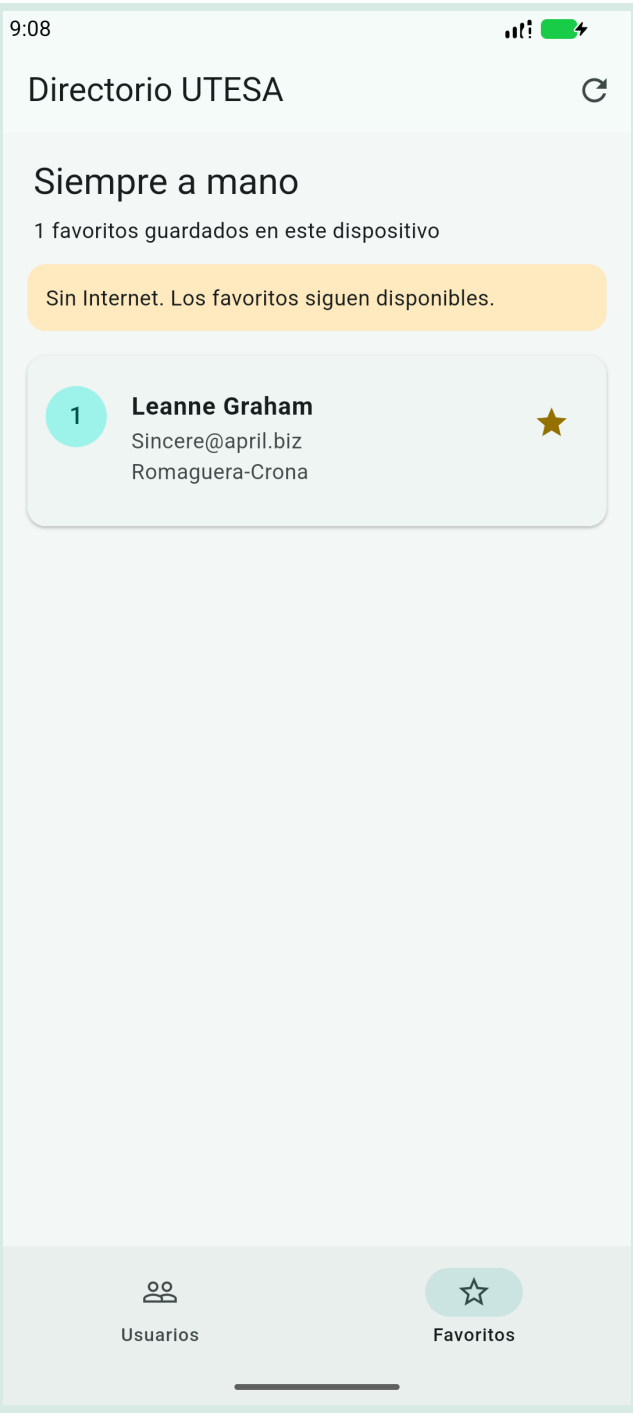
Favoritos: usuarios guardados localmente.

Prueba sin conexión

Reinicio de la app con Wi-Fi y datos móviles desactivados



Sin Internet: aviso y reintento.



Los favoritos permanecen después del reinicio.

Código de consumo de la API

Fragmento exacto de lib/data/repository.dart

```
import 'dart:async';
import 'dart:convert';

import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

import 'models.dart';

class ApiException implements Exception {
  final int status;
  ApiException(this.status);
}

class DirectoryApi {
  final http.Client client;
  DirectoryApi(this.client);
  Future<List<dynamic>> _get(String path, [Map<String, String>? query]) async {
    final uri = Uri.https('jsonplaceholder.typicode.com', path, query);
    final response = await client.get(uri).timeout(const Duration(seconds: 15));
    if (response.statusCode != 200) throw ApiException(response.statusCode);
    return jsonDecode(utf8.decode(response.bodyBytes)) as List<dynamic>;
  }

  Future<List<User>> users() async =>
      (await _get('/users'))
          .map((j) => User(j as Map<String, dynamic>))
          .toList();
  Future<List<Post>> posts(int id) async => (await _get('/posts', {
    'userId': '$id',
  })).map((j) => Post.fromJson(j as Map<String, dynamic>)).toList();
  void close() => client.close();
}
```

users() realiza GET /users. posts(id) agrega userId como parámetro de consulta. Uri.https forma la URL segura y http.Client ejecuta la petición. Se verifica el estado y se convierte el arreglo JSON a modelos. El cliente se cierra cuando se libera el controlador.

Código de persistencia

Fragmento exacto de lib/data/repository.dart

```
// Los favoritos son datos públicos, pequeños y no críticos. Se guarda
// el usuario completo, no solamente su ID, para poder abrirlo sin red.
class FavoritesStore {
  final SharedPreferencesAsync prefs;
  FavoritesStore(this.prefs);
  Future<List<User>> read() async {
    final raw = await prefs.getString('favorites_v1');
    if (raw == null) return [];
    return (jsonDecode(raw) as List)
        .map((j) => User(j as Map<String, dynamic>))
        .toList();
  }

  Future<void> write(List<User> users) => prefs.setString(
    'favorites_v1',
    jsonEncode(users.map((u) => u.json).toList()),
  );
}

class DirectoryRepository {
  final DirectoryApi api;
  final FavoritesStore store;
  DirectoryRepository(this.api, this.store);
}
```

Actualización del favorito

```
Future<void> toggle(User u) async {
    if (saving) return;
    saving = true;
    storageError = null;
    emit();
    final next = [...favorites];
    if (isFavorite(u)) {
      next.removeWhere((f) => f.id == u.id);
    } else {
      next.add(u);
    }
    try {
      await repository.store.write(next);
      favorites = next;
    } catch (_) {
      storageError = 'No se pudo guardar el cambio. Inténtalo de nuevo.';
    } finally {
      saving = false;
      emit();
    }
  }
```

La interfaz cambia el favorito después de completar la escritura. La bandera saving evita escrituras simultáneas desde varios toques. Los errores se muestran mediante storageError. SharedPreferencesAsync se usa para preferencias y datos no críticos; no se utiliza para credenciales.

Validación y conclusiones

Ejecución reproducible y alcance de la entrega

Validación

flutter analyze: sin incidencias. flutter test: cuatro pruebas aprobadas, que verifican el mapeo de usuarios, el filtro userId, el manejo de HTTP 500 y el rechazo de JSON inválido. La prueba HTTP 500 utiliza un cliente simulado; no se modifica el servicio público.

Las capturas documentan carga, lista correcta, detalle con publicaciones, favoritos y comportamiento sin Internet. La evidencia de favoritos sin conexión se obtiene después de cerrar completamente y volver a abrir la app con las redes del emulador desactivadas.

Cómo ejecutar

```
cd Tarea5
flutter pub get
flutter analyze
flutter test
flutter run
# Generar APK instalable de desarrollo:
flutter build apk --debug
```

Abrir Tarea5 en Android Studio con Flutter y Dart instalados. Conectar un dispositivo o iniciar un emulador. El APK de desarrollo se genera en build/app/outputs/flutter-apk/app-debug.apk. No constituye una publicación firmada para Play Store.

Conclusiones

1. Separar presentación, estado y servicios de datos facilita cambiar la API o la persistencia y permite probar el consumo HTTP sin depender de la red real.

2. Una experiencia sin conexión requiere persistir los datos necesarios, no solo las referencias. Guardar usuarios completos permite consultar favoritos tras reiniciar la app.

3. Carga, error y ausencia de red son estados normales de una app conectada. Capturarlos y ofrecer reintentos evita cierres por excepciones de red y explica al usuario qué puede hacer.

4. El almacenamiento debe elegirse según volumen y sensibilidad. Los favoritos públicos pequeños caben en preferencias; un catálogo extenso requiere una base local y los tokens requieren almacenamiento seguro.

Fuentes técnicas

JSONPlaceholder: guía y recursos

Flutter: obtener datos con http

shared_preferences: API asíncrona y límites

Flutter: ciclo de vida de State